Student notes — Session 10 · Search, min-max and bubble sort

Slides: [`Class_10_Search_min_max_bubble_sort.pdf`](../slides/Class_10_Search_min_max_bubble_sort.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Search, min-max and bubble sort

The programs of this class are on Blackboard: Week 5 module, *Class 10 code*. It is a zip file. Unzip it and open the folder in Visual Studio Code. Every file has a comment at the top: what the program does, how to run it, and a list called **Try** with small changes to make. Make them.

How to run a program. Same as every week. In the terminal type `gcc -Wall name.c -o name` and press Enter, then `./name` (or `.\name` in Windows PowerShell) and press Enter.

Today you learn four jobs that programmers do with arrays all the time: find a value, find the smallest and the largest, swap two elements, and sort the whole array.

Slide 2 — What Tuesday gave us

What you need from Tuesday:

- An array holds many values of one type under one name: `int a[5] = {3, 1, 4, 1, 5};`
- The first element is `a[0]`. The last one is `a[4]`, not `a[5]`.
- `for (int i = 0; i < N; i++)` visits every element.
- C does not check the index. `a[5]` gives no error, just a wrong value.

Slide 3 — Recap: what is on the screen?

Try it before you look at the next slide: what are the two lines?

Slide 4 — Recap: what is on the screen?

`a[0]` is 3 and `a[4]` is 5, so the first line is `8`.

The loop adds 3 + 1 + 4 + 1 + 5, so the second line is `sum = 14`.

File: `01_recap_array.c`.

Slide 5 — Today's roadmap

Four parts: arrays and functions, search, smallest and largest, swap and bubble sort.

Slide 6 — Arrays and functions

A function can take a whole array as a parameter.

Slide 7 — A function that prints an array

```
void print_array(int a[], int n)
```

- `int a[]` is the array. The brackets are empty, so the function works for an array of any size.
- `int n` is how many elements the array has.

Inside, the loop is the one you know: `i` from 0 to `n - 1`.

Slide 8 — The call: the name, and the size

To call the function, write the **name** of the array, without brackets, and then its size:

```
print_array(primes, 5);
print_array(ages, 3);
```

Output:

```
2 3 5 7 11
18 19 21
```

Wrong: `print_array(primes[5], 5)` or `print_array(primes[], 5)`. File: `02_print_array.c`.

Slide 9 — Why send the size?

An array in C does not know its own length. Inside the function, `a` is only "an array of `int`". So you must send the length as a second argument, `n`. Every array function today has this shape: the array first, then `n`.

Slide 10 — A function that returns the sum

`sum_array` adds the elements and gives back one `int`, the sum. The total starts at 0 before the loop, and `return sum;` comes after the loop.

Slide 11 — Using sum_array

Five marks: 7, 9, 6, 10, 8. `sum_array(marks, N)` returns 40, and the average is `(double) total / N`:

```
sum = 40
average = 8.00
```

File: `03_sum_array.c`.

Slide 12 — Predict: does the array change?

In class 8, `double_it(x)` could not change `x`: the function got a copy. Here `double_all` doubles every element of an array. Predict the two lines before you look at the next slide.

Slide 13 — Run it: the array DID change

```
before: 1 2 3
after:  2 4 6
```

The array in `main` changed. A function that gets an array works on the caller's own elements, not on a copy. File: `04_double_all.c`.

Slide 14 — An int and an array

- An `int` parameter is a **copy**. `double_it(x)` leaves `x` at 5.
- An array parameter is **not** a copy. `double_all(a, 3)` changes `a` to 2 4 6.

Slide 15 — Arrays are not copied

Rule: if a function changes `a[i]`, `main` sees the change. You will learn why when we reach pointers, after the midterm. Today this is useful: a function can sort your array.

Slide 16 — Search

Search answers two questions: is the value in the array, and if it is, where?

Slide 17 — Linear search

Linear search: look at every element, one by one, from the left. If it is the value you want, remember its index and stop. If you reach the end, the value is not there.

Slide 18 — Search for 23, step by step

We look for 23 in `4 8 15 16 23 42`. Before the loop, `pos` is `-1`. It means "not found yet". -1 is never a real index, because indexes start at 0.

Slide 19 — Search for 23, step by step

`i`	`a[i]`	equal to 23?	`pos`
0	4	no	-1
1	8	no	-1
2	15	no	-1
3	16	no	-1

Slide 20 — Search for 23, step by step

At `i = 4`, `a[4]` is 23. `pos` becomes 4 and `break` stops the loop. There is no need to look at 42.

Slide 21 — After the loop: found or not?

After the loop, check `pos`. If it is still -1, the value was not found.

```
23 found at index 4
```

File: `05_search.c`.

Slide 22 — Predict: a value that is not there

Change the target to 7. What is printed? Answer on the next slide.

Slide 23 — Predict: a value that is not there

```
7 not found
```

The loop checks all six elements. None is 7, so `pos` stays -1.

Slide 24 — Search as a function

```
int find(int a[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (a[i] == target) {
            return i;
        }
```

```
        }
        return -1;
    }
```

- `return i;` inside the loop: found. `return` ends the function at once, so no `break` is needed.
- `return -1;` **after** the loop: only then have you looked at every element.

`find(a, 6, 16)` gives 3 and `find(a, 6, 7)` gives -1. File: `06_find.c`.

Slide 25 — Careful: one small else

This `find` has an `else` inside the loop. Try to answer before the next slide: what does `find(a, 6, 16)` return?

Slide 26 — find 16 says -1

```
find 4: 0
find 16: -1
```

16 is in the array, but `find` says -1. At `i = 0`, `a[0]` is 4, which is not 16, so the `else` runs `return -1` and the function ends. Only the first element is ever checked. File: `07_bug_find_else.c`.

Slide 27 — gcc saw something too

`gcc -Wall` warns:

```
warning: control reaches end of non-void function [-Wreturn-type]
```

It means the function can reach its last `}` without a `return`. The fix: delete the `else`, and put `return -1;` after the loop.

Slide 28 — Predict: count the sixes

The rolls are `3 6 2 6 1 6 4 2 5 6`. How many sixes, how many twos? Answer on the next slide.

Slide 29 — Counting: no break

```
sixes: 4
twos: 2
```

The counter `c` starts at 0 and `c++` is inside the `if`. There is no `break`: to count, you look at every element. File: `08_count.c`.

Slide 30 — Search or count?

- **Search**: where is it? Stop at the first one. Return the index, or -1.
- **Count**: how many? Look at all of them. `c++` inside the `if`.

Slide 31 — Smallest and largest

Now: the smallest and the largest value in an array.

Slide 32 — The best so far

Take the first element as the best so far. Look at each of the others. If one is better, it is the new best. At the end, the best so far is the best of all.

Slide 33 — Largest of 12, 45, 7, 30, 19

`max` starts as `a[0]`, which is 12. Because `a[0]` is already used, the loop starts at `i = 1`.

Slide 34 — Largest of 12, 45, 7, 30, 19

`a[1]` is 45. 45 > 12, so `max` becomes 45.

Slide 35 — Largest of 12, 45, 7, 30, 19

`i`	`a[i]`	bigger than `max`?	`max`
start			12
1	45	yes	45
2	7	no	45
3	30	no	45
4	19	no	45

It prints `max = 45`. File: `09_max.c`.

Slide 36 — Careful: start from 0?

Here `max` starts at 0, and all the temperatures are below zero. Predict the output before the next slide.

Slide 37 — Start from 0: wrong

```
warmest: 0
```

0 is not in the array. No temperature is bigger than 0, so `max` never changes. The fix is `int max = temps[0];`, and then it prints `warmest: -2`. gcc gives no warning for this mistake, so you must check it yourself. File: `10_bug_max_zero.c`.

Slide 38 — The rule

Start the best so far at `a[0]`, never at a number you chose. 0 is wrong when all values are negative; 100 is wrong when all values are above 100. `a[0]` is always a real value from the array.

Slide 39 — Smallest: flip one sign

The smallest uses the same code with `<` instead of `>`:

```
if (a[i] < min) {
    min = a[i];
}
```

Slide 40 — min_of and max_of in main

A week of temperatures, `14 17 11 19 16 9 13`:

```
coldest: 9
warmest: 19
```

File: `11_min_max.c`.

Slide 41 — Where is the smallest?

To know **where** the smallest is, keep its index in `pos`, not its value. `pos` starts at 0, and you compare with `price[pos]`.

Slide 42 — Where is the smallest?

`i`	`price[i]`	smaller than `price[pos]`?	`pos`
start			0
1	9	yes	1
2	7	yes	2
3	11	no	2
4	8	no	2

It prints `cheapest: index 2, price 7`. With the index you also have the value: `price[pos]`. File: `12_min_position.c`.

Slide 43 — Swap and bubble sort

To sort, you first need to swap two elements.

Slide 44 — Careful: swap two elements

`a[0]` is 3 and `a[1]` is 7. The program tries to swap them. Predict before the next slide.

Slide 45 — Both are 7

```
7 7
```

`a[0] = a[1];` puts 7 into `a[0]`, and the 3 is lost. Then `a[1] = a[0];` copies that 7 back. File: `13_bug_swap_no_temp.c`.

Slide 46 — Swap with a third box

```
int temp = a[0];
a[0] = a[1];
a[1] = temp;
```

It prints `7 3`. `temp` keeps the 3 safe while `a[0]` is overwritten. File: `14_swap.c`.

Slide 47 — A swap in three moves

1. Save: `temp = a[0];` (temp is 3)
2. Copy: `a[0] = a[1];` (a[0] is 7)
3. Restore: `a[1] = temp;` (a[1] is 3)

Like swapping the drinks in two full glasses: you need a third, empty glass.

Slide 48 — Bubble sort: the idea

Compare two neighbours. If they are in the wrong order, swap them. Do this for every pair from left to right: that is one **pass**. After one pass the largest number is at the end. Repeat the pass until the array is in order.

Slide 49 — One pass over 5 3 8 4 2

5 and 3 are in the wrong order: swap. The array is now `3 5 8 4 2`.

Slide 50 — One pass over 5 3 8 4 2

5 and 8 are in the right order: leave them.

Slide 51 — One pass over 5 3 8 4 2

compare	swap?	array after
5 and 3	yes	3 5 8 4 2
5 and 8	no	3 5 8 4 2
8 and 4	yes	3 5 4 8 2
8 and 2	yes	3 5 4 2 8

The 8 was carried to the end. It is in its place and will not move again.

Slide 52 — One pass in C

```
for (int j = 0; j < N - 1; j++) {
    if (a[j] > a[j + 1]) {
        int temp = a[j];
        a[j] = a[j + 1];
        a[j + 1] = temp;
    }
}
```

The condition is `j < N - 1` because the loop uses `a[j + 1]`. With `j < N`, the last comparison would read `a[N]`, outside the array. It prints `3 5 4 2 8`. File: `15_one_pass.c`.

Slide 53 — Pass after pass

After pass 1, the largest number, 8, is in its place.

Slide 54 — Pass after pass

Pass 2 does the same for the rest: 5 moves next to the 8.

Slide 55 — Pass after pass

pass	array after it	in place
start	5 3 8 4 2	
1	3 5 4 2 8	8
2	3 4 2 5 8	5 8
3	3 2 4 5 8	4 5 8
4	2 3 4 5 8	all

5 numbers need 4 passes. In general, N numbers need N - 1 passes.

Slide 56 — Bubble sort: two loops

The outer loop counts the passes: `pass` from 1 to N - 1. The inner loop walks the pairs, like the one pass on slide 52.

Slide 57 — Why j < N - pass?

After pass 1 the last element is in place, after pass 2 the last two, and so on. So each pass can stop one pair earlier: `j < N - pass`. With `j < N - 1` the array is still sorted correctly; the program only does comparisons it does not need.

Slide 58 — Run it: watch every pass

```
pass 1: 3 5 4 2 8
pass 2: 3 4 2 5 8
pass 3: 3 2 4 5 8
pass 4: 2 3 4 5 8
```

The same rows as the table on slide 55. File: `16_bubble_sort.c`.

Slide 59 — Predict: four numbers

Bubble sort on `4 3 2 1`: how many passes, and what is the array after pass 1? Try it on paper first.

Slide 60 — Predict: four numbers

4 numbers need 3 passes. Pass 1 carries the 4 to the end: `3 2 1 4`. Pass 2 gives `2 1 3 4`, and pass 3 gives `1 2 3 4`.

Slide 61 — Sort as a function

`void bubble_sort(int a[], int n)` returns nothing. It does not need to: it changes the caller's array, like `double_all` on slide 13.

Slide 62 — Sorting eight marks

```
72 95 61 88 79 54 90 67
54 61 67 72 79 88 90 95
```

After `bubble_sort(marks, N);` the array in `main` is in order. File: `17_sort_function.c`.

Slide 63 — Extra: stop when nothing moves

An extra idea. If a whole pass makes no swap, the array is already sorted, so you can stop. `swapped` is 0 at the start of each pass and becomes 1 when a swap happens. After the pass, `if (!swapped) break;`.

For `1 2 3 5 4` it prints `passes: 2`: pass 1 swaps 5 and 4, pass 2 swaps nothing. File: `18_bubble_early_exit.c`.

Slide 64 — Sorting high to low

To sort from high to low, change `>` to `<`:

```
if (a[j] < a[j + 1])
```

Now the smaller number moves to the end.

Slide 65 — Your turn

Two small functions to write on paper or in VS Code.

Slide 66 — Your turn: index_of_max

Write `int index_of_max(int a[], int n)`. It returns the index of the largest element. For `1 0 3 4 2 1` it returns 3. Hint: it is like the cheapest shop on slide 42, with `>` instead of `<`.

Slide 67 — index_of_max: one answer

```
int index_of_max(int a[], int n) {
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (a[i] > a[pos]) {
            pos = i;
        }
    }
    return pos;
}
```

It prints `best match: index 3`. File: `19_task_index_of_max.c`.

Slide 68 — Your turn: contains

Write `int contains(int a[], int n, int x)`. It returns 1 if `x` is in the array and 0 if it is not. Where does `return 0;` go?

Slide 69 — contains: one answer

`return 1;` goes inside the loop, as soon as `x` is found. `return 0;` goes **after** the loop, the same rule as `return -1` in `find`. With input 13 the program prints `yes, lucky`; with 5 it prints `no`. File: `20_task_contains.c`.

Slide 70 — Today in one slide

- Send an array and its size: `f(a, n)`.
- A function can change the caller's array.
- Search: return the index, or -1 after the loop.
- Min and max: start from `a[0]`.
- Swap with `temp`. Bubble sort: passes of neighbour swaps.

Slide 71 — Before Tuesday

- **Knowledge check 10** on Blackboard, on this class.
- **Project 1** (Blackjack) is due **Sunday 11 October, 23:59**. Upload early.
- **Quiz 3** is on **Friday 16 October** and covers classes 7 to 10: functions, scope, call-by-value, recursion and arrays.
- This class's code: Blackboard, Week 5 module, *Class 10 code*.

Next class: characters and strings. A string is an array of `char`, so everything from today comes back.